

Assignment 1

Basic Graphics and Immediate Mode GUI

Computer Graphics Course

C++ | CLion | MiniFB | MicroUI

Student 1	Natalie Bittar ID:212586044
Student 2	Adan Awad ID:32430267
Course	Computer Graphics
Assignment	Basic Graphics and Immediate Mode GUI

This report documents the design, implementation, testing, and results of the interactive graphics application developed for Assignment 1.

Report Structure

- Part 1: Manipulating the Framebuffer
- Part 2: Immediate Mode UI Declaration
- Part 3: Real-Time Graphics Loop and Input Handling
- Part 4: UI Architecture and Renderer Bridge
- Part 5: Binding UI to Application State
- Part 6: Interactive Line Drawing Application
- Part 7: Pair Programming Extensions
- Conclusion

Part 1: Manipulating the Framebuffer

The framebuffer is represented by a one-dimensional array in which each 32-bit value stores the RGB color of one pixel:

```
static uint32_t g_buffer[WIDTH * HEIGHT];
```

For each array index i , the corresponding two-dimensional pixel coordinates are calculated as follows:

```
int x = i % WIDTH;  
int y = i / WIDTH;
```

A creative pattern was generated by combining a checkerboard pattern with radial distance from the center of the screen. The distance value was calculated using the Euclidean distance formula:

```
int dx = x - center_x;  
int dy = y - center_y;  
int distance = static_cast<int>(  
    std::sqrt(static_cast<double>(dx * dx + dy * dy)));
```

The checkerboard depends on both x and y , so the result is a true two-dimensional pattern rather than a solid color or a one-dimensional strip:

```
int checker = ((x / cell_size) + (y / cell_size)) % 2;
```

The final red, green, and blue values depend on the pixel position, the distance from the center, and the checkerboard state. This creates a colorful combination of concentric forms, gradients, and alternating cells.

Part 2: Immediate Mode UI Declaration

A custom button was added to the MicroUI window. The button toggles an external Boolean variable and also prints a message to the console:

```
if (mu_button(ctx, "Show / Hide Message")) {  
    show_message = !show_message;  
    printf("Custom button clicked\n");  
}
```

The label is declared every frame, but it is displayed only when the external state variable is true:

```
if (show_message) {  
    mu_label(ctx, "The custom message is visible.");  
}
```

This demonstrates the Immediate Mode architecture: widgets do not permanently store application state. Instead, they read and modify external variables.

Part 3: Real-Time Graphics Loop and Input Handling

The MiniFB character callback was modified so that pressing the English letter E toggles a background effect:

```
if (character == 'e' || character == 'E') {
    g_invert_effect = !g_invert_effect;
    printf("Background effect toggled\n");
    return;
}
```

The return statement consumes the event, so the character E is not passed to the textbox. All other characters are forwarded to the standard MicroUI input bridge.

```
ui_bridge_char_input(window_pointer, character);
```

Part 4: UI Architecture and Renderer Bridge

MicroUI calculates the logical layout and hit-testing rectangles, while UIRenderer converts the abstract drawing commands into framebuffer pixels. To demonstrate the separation, the visual GUI was temporarily shifted by 40 pixels to the right and 20 pixels downward.

```
const int visual_offset_x = 40;
const int visual_offset_y = 20;
```

Only the rendered pixels were moved. The logical interaction rectangles remained in their original positions. Therefore, clicking the visible button did not activate it. The cursor had to be placed 40 pixels to the left and 20 pixels above the visible button.

After the experiment, the renderer was restored to its original coordinates so the visual and logical positions matched again.

Part 5: Binding UI to Application State

The following variables were introduced to control the background pattern:

```
static float pattern_size = 70.0f;
static float red_shift = 0.0f;
static float green_shift = 0.0f;
static float blue_level = 180.0f;
```

Each variable was bound to a MicroUI slider by passing its memory address:

```
mu_slider(ctx, &pattern_size, 20, 150);
mu_slider(ctx, &red_shift, 0, 255);
mu_slider(ctx, &green_shift, 0, 255);
mu_slider(ctx, &blue_level, 0, 255);
```

- Pattern Size changes the dimensions of the checkerboard cells.
- Red Shift changes the red component of the background.
- Green Shift changes the green component of the background.
- Blue Level changes the blue intensity.

Because the rendering loop is executed continuously, every slider modification is visible immediately.

Part 6: Interactive Line Drawing Application

Bresenham Line Algorithm

Bresenham's algorithm was implemented using integer arithmetic. The same function supports horizontal, vertical, diagonal, steep, shallow, forward, and reversed lines.

```
void draw_line_bresenham(int x0, int y0, int x1, int y1, uint32_t color) {
    int dx = std::abs(x1 - x0);
    int sx = x0 < x1 ? 1 : -1;
    int dy = -std::abs(y1 - y0);
    int sy = y0 < y1 ? 1 : -1;
    int error = dx + dy;

    while (true) {
        put_pixel(x0, y0, color);
        if (x0 == x1 && y0 == y1) break;
        int error2 = 2 * error;
        if (error2 >= dy) { error += dy; x0 += sx; }
        if (error2 <= dx) { error += dx; y0 += sy; }
    }
}
```

UX Planning

Two interaction methods were considered:

Method 1: Click Once, Then Click Again

- Advantages: simple implementation and no need to hold the mouse button.
- Disadvantages: the user may forget whether the first point was already selected, and canceling an unfinished line requires extra logic.

Method 2: Click, Drag, and Release

- Advantages: natural interaction, live preview, and clear start/end points.
- Disadvantages: requires tracking press, drag, and release states.

The click-drag-release method was selected because it is more intuitive and is similar to common drawing applications.

Permanent Lines and Live Preview

Saved lines are stored in a vector. A temporary preview is drawn while the mouse button is held. The line is inserted into the vector only when the button is released, preventing many temporary lines from being stored during one drag operation.

```
static std::vector<Line> saved_lines;
```

- RGB sliders select the current shape color.
- Multiple lines can be stored permanently.
- A Clear Canvas button removes all shapes.
- Drawing is restricted to the right side of the screen so it does not interfere with the GUI.

Part 7: Pair Programming Extensions

Extension 1: Bresenham Circle Algorithm

The circle algorithm uses eight-way symmetry. For every calculated point in one octant, seven reflected points are drawn:

```
put_pixel(center_x + x, center_y + y, color);
put_pixel(center_x - x, center_y + y, color);
put_pixel(center_x + x, center_y - y, color);
put_pixel(center_x - x, center_y - y, color);
put_pixel(center_x + y, center_y + x, color);
put_pixel(center_x - y, center_y + x, color);
put_pixel(center_x + y, center_y - x, color);
put_pixel(center_x - y, center_y - x, color);
```

The mouse press defines the circle center. The distance from the center to the current mouse position defines the radius:

```
int dx = preview_x - shape_start_x;
int dy = preview_y - shape_start_y;
int radius = static_cast<int>((
    std::sqrt(static_cast<double>(dx * dx + dy * dy))));
```

The GUI allows switching between Line Mode and Circle Mode. Circles also have a live preview and are stored permanently after mouse release.

Extension 2: Performance Profiling

A naive floating-point line algorithm was implemented for comparison. It calculates an x increment and a y increment, repeatedly adds them, and rounds the coordinates to the nearest pixel.

```
float x_increment = static_cast<float>(dx) / steps;
float y_increment = static_cast<float>(dy) / steps;
```

The benchmark generates 100,000 random lines and renders the same line set using both algorithms. Execution time is measured using `std::chrono::high_resolution_clock`.

Run	Bresenham	Naive Float
1	194.709 ms	563.782 ms
2	231.813 ms	574.729 ms

In both runs, Bresenham was significantly faster. It completed the benchmark in approximately 195-232 ms, while the naive method required approximately 564-575 ms. In this environment, Bresenham was about 2.5 to 3 times faster.

Although modern processors handle floating-point arithmetic efficiently, the naive method still performs repeated floating-point additions and rounding for every pixel. Bresenham relies on integer additions, comparisons, and error updates.

Extension 3: Xiaolin Wu Anti-Aliasing

Bresenham lines may appear jagged because each step selects one full-intensity pixel. Xiaolin Wu's algorithm draws two neighboring pixels and distributes intensity according to the fractional position of the ideal mathematical line.

```
plot_wu_pixel(steep, x, integer_y, color,
    reverse_fractional_part(intersection_y));
```

```
plot_wu_pixel(steep, x, integer_y + 1, color,  
              fractional_part(intersection_y));
```

Manual Blending Without Alpha

The assignment ignores the alpha channel, so the anti-aliased pixel cannot be stored as a transparent color. Instead, the source color is manually blended with the destination color already present in the framebuffer:

```
result = source * intensity + destination * (1 - intensity)
```

For example, the red component is calculated as follows:

```
uint8_t result_r = static_cast<uint8_t>(  
    source_r * intensity +  
    destination_r * (1.0f - intensity));
```

This also addresses drawing one anti-aliased line over another: the new color is blended with the existing pixel rather than completely replacing it. Because the background and saved shapes are redrawn from scratch every frame, opacity does not accumulate uncontrollably between frames.

Switching Between Algorithms

A GUI button switches between Bresenham and Xiaolin Wu. The button text changes according to the current algorithm, and the current mode is also displayed. Existing saved lines are redrawn using the selected algorithm, enabling immediate visual comparison.

Conclusion

This assignment combined low-level framebuffer manipulation, mathematical rasterization algorithms, input handling, and Immediate Mode GUI state into a complete interactive drawing application.

- Converted one-dimensional framebuffer indices into two-dimensional coordinates.
- Generated mathematical two-dimensional background patterns.
- Created and controlled Immediate Mode GUI widgets.
- Intercepted and consumed keyboard events.
- Demonstrated the separation between GUI hit-testing and visual rendering.
- Bound sliders to external application state.
- Implemented interactive Bresenham line drawing with live preview.
- Implemented Bresenham circle drawing using eight-way symmetry.
- Benchmarked Bresenham against a naive floating-point implementation.
- Implemented Xiaolin Wu anti-aliasing and manual RGB blending.

The final program supports multiple permanent lines and circles, color controls, clear functionality, live previews, real-time background modification, performance testing, and instant switching between aliased and anti-aliased line rendering.